

LQCD Master

格点 QCD 自主 AI 科学家框架使用手册

张鑫

2026 年 7 月 20 日

摘要

LQCD Master 是一个面向格点量子色动力学 (Lattice QCD) 的自主 AI 科学家框架。它将自然语言描述的物理任务自动翻译为可执行的 PyQUDA GPU 计算程序及 Slurm 集群提交脚本，并通过 Planner-Executor 双阶段流水线实现从物理需求到集群计算的端到端自动化。本手册详细介绍 LQCD Master 的架构设计、安装配置、命令行使用、技能系统、基准测试以及核心 API 接口。

目录

1	项目概述	2
1.1	主要特性	2
1.2	流水线概览	2
2	项目结构	2
3	安装	4
3.1	环境要求	4
3.2	安装步骤	4
3.3	环境配置 (.env)	4
4	配置	5
4.1	集群与系综配置 (configs/)	5
4.1.1	配置文件结构	5
4.2	技能路由配置 (configs/skills.yaml)	5
5	核心架构	6
5.1	WorkflowOrchestrator (工作流编排器)	6
5.2	PlannerAgent (规划器)	6

目录	2
5.3 ExecutorAgent (执行器)	7
5.4 SkillRunner (技能运行器)	7
6 命令行使用	8
6.1 基本用法	8
6.2 命令行参数完整列表	9
7 示例演示	9
7.1 示例 1: 计算 介子两点关联函数	9
7.2 示例 2: 批量运行脚本	10
8 技能系统	10
8.1 技能模块结构	10
8.2 技能路由机制	11
9 基准测试	11
9.1 测试任务概览	11
9.2 实验结果	12
10 API 接口清单	12
10.1 LQCDLLMClient	12
10.2 WorkflowOrchestrator	13
10.3 PlannerAgent	14
10.4 ExecutorAgent	15
10.5 BuiltinToolClient	16
10.6 SlurmSubmitTool	16
10.7 SkillRegistry / SkillRouter / SkillRunner	16
10.8 辅助工具模块	17
10.9 Wick 收缩生成器	18
11 依赖关系	19
12 开发与贡献	19
12.1 自定义集群配置	19
12.2 添加新技能	19
12.3 扩展基准测试	20

1 项目概述

LQCD Master 的核心目标是将自然语言描述的格点 QCD 物理请求自动转换为可执行的 PyQUDA 程序和 Slurm 提交脚本。系统结合了 **Planner** (规划器) 和 **Executor** (执行器) 两个智能体，形成一条可复现、有人类监督的计算流水线。

1.1 主要特性

- **自然语言接口**：用户只需用中文或英文描述物理任务（如”计算 介子两点关联函数”），系统自动生成计算方案和代码
- **双阶段流水线**：Planner 负责物理方案生成（含评审-重写循环），Executor 负责代码生成（含静态分析和自动调试）
- **人在回路**：每个阶段都设有人工检查点，系统不会在未经确认的情况下向集群提交作业
- **自动调试**：若静态分析检测到生成代码中的错误，Executor 会自动重写（上限为 `executor_static_check_rounds` 轮）
- **技能路由**：领域知识（关联函数理论、PyQUDA API、分析方法）根据当前任务相关性注入 LLM 提示词
- **全面基准测试**：70 个独立验证任务，覆盖 5 类可观测量

1.2 流水线概览

自然语言任务 → **Planner** (方案生成 + 评审 + 重写) → [人工检查点] → **Executor** (代码生成 + 自动调试) → [人工检查点] → **Slurm 提交** → GPU 计算结果

图 1: LQCD Master 流水线示意图

2 项目结构

```

1  .
2      run.py                # CLI入口点
3      requirements.txt      # Python依赖
4      .env.template         # 环境变量模板

```

```

5     core_architecture/          # Planner → Executor → Submit 流
    水线
6         orchestrator.py        #  workflow编排器
7         planner.py             #  规划器Agent
8         executor.py            #  执行器Agent
9     utils/                     #  LLM客户端、工具、I/O、技能系统
10        llm_client.py           #  OpenAI兼容LLM客户端
11        tool_client.py          #  内置工具客户端（搜索/解析）
12        submit_tool.py         #  Slurm提交工具
13        config_utils.py        #  配置加载工具
14        io_utils.py            #  文件I/O工具
15        prompt_loader.py        #  提示词加载器
16        skill_utils.py          #  技能系统核心
17        static_check.py         #  静态代码分析
18        generate_einsum/        #  Wick收缩与einsum生成
19    prompts/                   #  各阶段提示词模板
20        planner/                #  solve / critique / rewrite
21        executor/              #  generate / critique / rewrite
22    configs/                   #  集群、系综、技能配置
23        config.yaml            #  Slurm/Runtime配置
24        ensemble_presets.yaml  #  系综预设
25        skills.yaml            #  技能路由配置
26    skills/                    #  领域知识模块
27        lqcd-analysis/         #  数据分析技能
28        lqcd-physics-spectrum/ #  能谱学技能
29        lqcd-physics-correlator/ #  关联函数理论
30        pyquda-tool/           #  PyQUDA API指南
31        pyquda-gauge/          #  规范场操作
32    benchmark/                 #  70个验证任务 + 标准方法参考数
    据
33    experiments/              #  实验结果（按模型分类）

```

Listing 1: LQCD Master 项目目录结构

3 安装

3.1 环境要求

表 1: 运行环境要求

组件	要求
Python	≥ 3.10
pip 依赖	openai, prompt_toolkit, PyYAML, numpy, opt_einsum
GPU 后端	CuPy (CUDA 或 ROCm)
格点 QCD	PyQUDA + QUDA (从源码编译安装)
集群	Slurm + GPU 分区
外部 API	OpenAI 兼容 LLM 端点 + Serper.dev (网页搜索)

3.2 安装步骤

```

1 # 1. 克隆仓库
2 git clone https://github.com/sjtu-sai-agents/LQCD_Master.git
3 cd LQCD_Master
4
5 # 2. 安装Python依赖
6 pip install -r requirements.txt
7
8 # 3. 配置环境变量
9 cp .env.template .env
10 # 编辑 .env 文件, 填入LLM API密钥和Serper密钥

```

Listing 2: LQCD Master 安装流程

3.3 环境配置 (.env)

```

1 OPENAI_API_KEY=sk-...           # LLM API密钥 (必需)
2 OPENAI_BASE_URL=...             # API端点 (默认: api.deepseek.com/
   v1)
3 OPENAI_MODEL=...                # 模型名称 (默认: deepseek-v4-pro)
4 SERPER_API_KEY=...              # Serper.dev密钥 (网页搜索工具)

```

Listing 3: .env 配置文件示例

4 配置

4.1 集群与系综配置 (configs/)

configs/config.yaml 定义了 Slurm 分区、模块加载、MPI 启动和 QUDA 环境路径。configs/ensemble_presets.yaml 定义了 7 个格点系综的预设信息，包括规范组态路径、夸克质量、clover 系数和多重网格参数。

4.1.1 配置文件结构

```

1 script:      # 脚本模板设置
2   shell: /bin/bash
3 slurm:       # Slurm作业调度参数
4   partition: gpu
5   nodes: 1
6   gres: gpu:4
7   ntasks_per_node: 4
8   ntasks_per_socket: 4
9 runtime:    # 运行时环境
10  modules: [cuda/12.0, openmpi/4.1]
11  exports: {QUDA_PATH: /path/to/quda}
12  activate: conda activate lqcd
13  source: [source ./env.sh]
14  launch: mpirun -np
15 ensemble:   # 系综配置
16   preset: C24P29
17   cfg_num: [10000, 10100, 10200]
18   process_grid: [2, 2, 1, 1]
19 measurement: # 测量参数
20 solver:      # 求解器参数
21 output:      # 输出设置
22 workflow:    # 工作流设置
23   executor_static_check_rounds: 5
24   executor_dynamic_test_rounds: 2

```

Listing 4: configs/config.yaml 结构

4.2 技能路由配置 (configs/skills.yaml)

控制哪些领域知识模块在每个流水线阶段被注入。系统内置了五个技能模块：

- **lqcd-physics-correlator**: 关联函数理论（介子、质子等强子谱学）
- **lqcd-physics-spectrum**: 能谱分析
- **lqcd-analysis**: 数据分析方法
- **pyquda-tool**: PyQUDA API 使用指南
- **pyquda-gauge**: 规范场操作

5 核心架构

5.1 WorkflowOrchestrator（工作流编排器）

WorkflowOrchestrator 是整个系统的主控制器，负责协调 Planner 和 Executor 两个子智能体，管理人工检查点，以及处理 Slurm 作业的提交和监控。

```

1  from core_architecture.orchestrator import WorkflowOrchestrator
2  from utils.llm_client import LQCDLLMClient
3  from utils.submit_tool import SlurmSubmitTool
4  from utils.tool_client import BuiltinToolClient
5
6  llm = LQCDLLMClient(dotenv_path=".env")
7  tool_client = BuiltinToolClient(dotenv_path=".env")
8  submit_tool = SlurmSubmitTool()
9
10 orchestrator = WorkflowOrchestrator(
11     task="Calculate the pion two-point correlator",
12     run_dir=Path("runs/demo"),
13     llm_client=llm,
14     tool_client=tool_client,
15     submit_tool=submit_tool,
16     test_mode=True,
17     non_interactive=False,
18 )
19 final_state = orchestrator.run()

```

Listing 5: WorkflowOrchestrator 使用示例

5.2 PlannerAgent（规划器）

Planner 通过三阶段流程将自然语言任务转化为结构化的物理计算方案：

1. **Solve**: 根据任务描述和系综信息生成初始计算方案 (YAML + Markdown)
2. **Critique**: 对方案进行评审, 识别问题和风险, 提出修订建议
3. **Rewrite**: 根据评审意见修订方案

输出格式包含:

- `plan_yaml`: 结构化计算方案 (含任务、系综、物理、测量、输出五个部分)
- `summary_md`: 人类可读的方案摘要
- `citations`: 验证过的引用 URL 列表

5.3 ExecutorAgent (执行器)

Executor 将 Planner 输出的计算方案转换为可执行的 Python 代码和 Slurm 脚本:

1. **Generate**: 生成 PyQUDA 主程序 + 测试提交脚本 + 完整提交脚本
2. **Static Analysis**: 静态分析生成的代码, 检测语法错误和导入问题
3. **Critique**: 结合静态分析结果评审代码质量
4. **Rewrite**: 根据评审意见和静态分析错误自动重写 (最多 `executor_static_check_rounds` 轮)

生成产物:

- `main.py`: PyQUDA 计算主程序
- `submit_test.sh`: 测试提交脚本 (小规模验证运行)
- `submit_full.sh`: 完整提交脚本 (生产运行)
- `static_analysis.json`: 静态分析报告
- `critique.json`: 代码评审报告

5.4 SkillRunner (技能运行器)

技能系统通过 SkillRunner 实现领域知识注入:

- **SkillRegistry**: 从 `skills/` 目录加载技能模块
- **SkillRouter**: 根据任务内容匹配最相关的技能
- **SkillMessageAssembler**: 将技能内容组装到 LLM 提示词中
- **ToolRegistry**: 注册和管理内置工具 (`web_search`, `web_parse`)

6 命令行使用

6.1 基本用法

```
1 # 交互式运行
2 python run.py
3
4 # 指定任务文本
5 python run.py --task "计算 介子两点关联函数, 使用C24P29系综和点源"
6
7 # 从文件读取任务
8 python run.py --task task.txt
9
10 # 测试模式 (仅生成代码, 不提交作业)
11 python run.py --task "..." --test
12
13 # 非交互模式 (自动接受所有检查点)
14 python run.py --task "..." --non-interactive
15
16 # 指定运行目录
17 python run.py --task "..." --run-dir runs/my_experiment
18
19 # 列出可用技能
20 python run.py --list-skills
```

Listing 6: LQCD Master 命令行接口

6.2 命令行参数完整列表

表 2: CLI 参数说明

参数	类型	说明
--task	str	任务描述文本或包含任务描述的文件路径
--test	flag	测试模式: 仅生成代码, 跳过 Slurm 提交
--run-dir	str	输出目录 (默认: runs/<timestamp>)
--non-interactive	flag	非交互模式: 自动接受所有检查点
--dotenv-path	str	自定义 .env 文件路径
--list-skills	flag	列出可用技能并退出

7 示例演示

7.1 示例 1: 计算 介子两点关联函数

```

1 # 使用非交互模式快速运行
2 python run.py --task \
3     "Calculate the pion two-point correlator on the C24P29 ensemble
4     using a point source at zero momentum." \
5     --test --non-interactive

```

Listing 7: 运行 介子两点关联函数计算

Planner 生成的方案 YAML 示例:

```

1 task: "Calculate pion two-point correlator"
2 ensemble:
3   preset: C24P29
4   cfg_num: [10000]
5   Lx: 24, Ly: 24, Lz: 24, Lt: 72
6   beta: 6.20
7 physics:
8   observable: pion_mass
9   hadron: pion
10  interpolator: gamma5

```

```

11 measurement:
12     method: distillation
13     nev: 100
14     mom_list: [[0, 0, 0, 0]]
15 output:
16     format: npy
17     path: ./output/
18     extras: {}

```

Listing 8: Planner 输出的 plan.yaml 示例

7.2 示例 2: 批量运行脚本

```

1  #!/bin/bash
2  # 批量计算多种强子的两点关联函数
3
4  TASKS=(
5      "Calculate pion two-point correlator on C24P29"
6      "Calculate kaon two-point correlator on C24P29"
7      "Calculate proton two-point correlator on C24P29"
8  )
9
10 for task in "${TASKS[@]"; do
11     python run.py --task "$task" \
12         --test --non-interactive \
13         --run-dir "runs/batch_$(date +%Y%m%d_%H%M%S)"
14 done

```

Listing 9: 批量运行实验脚本示例

8 技能系统

8.1 技能模块结构

每个技能模块位于 skills/目录下, 包含一个 SKILL.md 文件:

```

1 skills/
2     lqcd-analysis/SKILL.md           # 数据分析指导
3     lqcd-physics-spectrum/SKILL.md   # 强子能谱学
4     lqcd-physics-correlator/SKILL.md # 关联函数理论

```

```

5         reference/                                # 参考数据
6             pion_mass.md
7             proton_mass.md
8             rho_mass.md
9     pyquda-tool/SKILL.md                            # PyQUDA API使用指南
10        reference/
11            baryon_3pt_code.md
12    pyquda-gauge/SKILL.md                            # 规范场操作
13        reference/
14            wilson_loop.md

```

Listing 10: 技能模块目录结构

8.2 技能路由机制

技能路由根据以下规则将相关技能注入到各阶段的提示词中：

- **Planner 阶段**：选择与观测物类型（2pt、3pt、Wilson loop 等）匹配的技能
- **Executor 阶段**：选择与代码生成相关的 PyQUDA API 技能
- 技能注入优先级在 `configs/skills.yaml` 中配置

9 基准测试

9.1 测试任务概览

`benchmark/`目录定义了 70 个独立验证任务，涵盖 5 类可观测量，全部在 C24P29 系综（`cfg 10000`，点源，零动量）上评估。

表 3: 基准测试任务分类

可观测量	任务数	示例
2pt local	20	$\pi, K, \rho, D, D_s, J/\psi, p, \Lambda, \Xi, \Sigma, \Lambda_c, \Xi_c, \Omega_c$
2pt nonlocal	10	$\pi, K, \rho, D, D_s, J/\psi, D^*, D_s^*$ (Wilson 线位移 $z = 0..10$)
Wilson loop	12	$W(R,T), R,T \in \{1,2,3,4\}$, 平均于 XT, YT, ZT 平面
3pt meson	13	$D \rightarrow K/\pi, B \rightarrow D/K/\pi, K \rightarrow \pi, B_c \rightarrow J/\psi$
3pt baryon	15	$p \rightarrow p, \Lambda \rightarrow \Lambda, \Lambda_c \rightarrow \Lambda, \Lambda_b \rightarrow \Lambda_c$

9.2 实验结果

表 4: 70 任务套件上的交叉验证结果

实验	骨干模型	准确匹配	成功率
LQCD Master GPT-5.4	GPT-5.4	63	90.0%
LQCD Master DeepSeek	DeepSeek V4 Pro	56	80.0%

注：准确匹配定义为与标准方法在机器精度 ($|\Delta| < 10^{-12}$) 或相对误差 $< 10^{-3}$ 下一致。

10 API 接口清单

10.1 LQCDLLMClient

OpenAI 兼容的 LLM 客户端，支持自动加载 `.env` 配置。

```

1 class LQCDLLMClient:
2     """OpenAI 兼容客户端，自动加载 .env 配置"""
3
4     def __init__(
5         self,
6         *,
7         api_key: str | None = None,
8         base_url: str | None = None,
9         model: str | None = None,
10        max_retries: int | None = None,
11        dotenv_path: str | Path = ".env",
12        extra_body: dict[str, Any] | None = None,
13    ): ...
14
15    def chat(
16        self,
17        messages: list[dict[str, str]],
18        *,
19        stream: bool = False,
20        temperature: float | None = None,
21        max_tokens: int | None = None,
22        model: str | None = None,
23    ) -> str: ...

```

```

24
25     def chat_json(
26         self,
27         messages: list[dict[str, str]],
28         *,
29         stream: bool = False,
30         temperature: float | None = None,
31         max_tokens: int | None = None,
32         model: str | None = None,
33     ) -> dict[str, Any]: ...
34
35     def chat_json_with_tools(
36         self,
37         messages: list[dict[str, Any]],
38         *,
39         tools: list[dict[str, Any]],
40         tool_handler: Any,
41         max_turns: int = 8,
42         temperature: float | None = None,
43         max_tokens: int | None = None,
44         model: str | None = None,
45     ) -> tuple[dict[str, Any], list[dict[str, Any]]]: ...

```

Listing 11: LQCDLLMClient 接口

10.2 WorkflowOrchestrator

```

1  class WorkflowOrchestrator:
2      """工作流编排器 — Planner → Executor → Submit"""
3
4      def __init__(
5          self,
6          *,
7          task: str,
8          run_dir: Path,
9          llm_client: Any,
10         tool_client: Any,
11         submit_tool: Any,
12         test_mode: bool = False,
13         non_interactive: bool = False,

```

```

14         max_revision_rounds: int = 3,
15     ): ...
16
17     def run(self) -> dict[str, Any]: ...
18     # 返回完整状态字典: planner, executor, test_submission,
        full_submission

```

Listing 12: WorkflowOrchestrator 接口

10.3 PlannerAgent

```

1  class PlannerAgent:
2      """Planner Agent — 生成物理计算方案"""
3
4      def __init__(
5          self,
6          llm_client: Any,
7          tool_client: Any,
8          run_dir: Path,
9      ): ...
10
11     def run(self, task: str) -> dict[str, Any]: ...
12     # 返回: {plan_yaml, summary_md, citations}
13
14     def rewrite_with_feedback(
15         self,
16         task: str,
17         current: dict[str, Any],
18         feedback: str,
19         version: int
20     ) -> dict[str, Any]: ...
21
22     def validate_plan_coverage(self, plan_yaml: str) -> dict[str,
        Any]: ...
23     # 检查plan.yaml是否覆盖所有必需字段
24
25     def debug_state(self) -> dict[str, Any]: ...
26     # 返回内部状态 (solve/critique/rewrite payload, stage traces
        )

```

Listing 13: PlannerAgent 接口

10.4 ExecutorAgent

```

1  class ExecutorAgent:
2      """Executor Agent — 生成PyQUDA代码和Slurm脚本"""
3
4      def __init__(
5          self,
6          llm_client: Any,
7          submit_tool: Any,
8          run_dir: Path,
9          tool_client: Any | None = None,
10         max_static_check_rounds: int | None = None,
11     ): ...
12
13     def generate_bundle(
14         self,
15         *,
16         task: str,
17         planner_output: dict[str, Any],
18         version: int,
19         feedback: str | None = None,
20         previous_bundle: dict[str, Any] | None = None,
21     ) -> dict[str, Any]: ...
22     # 返回: {main_program_path, test_submit_path,
23         full_submit_path,
24         #         static_analysis_report_path, critique_report_path,
25         #         static_analysis, critique}
26
27     def submit_test(self, bundle: dict[str, Any], version: int)
28         -> dict[str, Any]: ...
29
30     def submit_full(self, bundle: dict[str, Any], version: int)
31         -> dict[str, Any]: ...
32
33     def collect_evidence_paths(self, bundle: dict[str, Any],
34         version: int) -> list[str]: ...

```


Listing 14: ExecutorAgent 接口

10.5 BuiltinToolClient

```

1 class BuiltinToolClient:
2     """内置工具客户端 — web_search 和 web_parse"""
3
4     def __init__(self, dotenv_path: str | Path = ".env"): ...
5
6     def web_search(self, query: str, num: int = 5) -> list[dict[
7         str, Any]]: ...
8     # 通过Serper API执行网页搜索
9
10    def web_parse(self, url: str) -> dict[str, Any]: ...
11    # 解析网页内容, 返回文本和元数据

```

Listing 15: BuiltinToolClient 接口

10.6 SlurmSubmitTool

```

1 class SlurmSubmitTool:
2     """Slurm作业提交工具"""
3
4     def submit(self, script_path: str) -> dict[str, Any]: ...
5     # 通过sbatch提交作业脚本
6     # 返回: {ok, job_id, stderr}

```

Listing 16: SlurmSubmitTool 接口

10.7 SkillRegistry / SkillRouter / SkillRunner

```

1 class SkillRegistry:
2     """技能注册表 — 从skills/目录加载"""
3     def __init__(self, skills_dir: Path): ...
4     def load(self) -> None: ...
5     def names(self) -> list[str]: ...
6     def get(self, name: str) -> Skill | None: ...
7

```

```

8 class SkillRouter:
9     """技能路由器 — 根据任务匹配最相关技能"""
10    def __init__(self, registry: SkillRegistry, config_path: Path
11        ): ...
12    def route(self, context: SkillContext) -> list[str]: ...
13
14 class SkillRunner:
15     """技能运行器 — 执行技能并管理LLM交互"""
16    def __init__(
17        self,
18        llm_client: Any,
19        registry: SkillRegistry,
20        router: SkillRouter,
21        assembler: SkillMessageAssembler,
22        tool_registry: ToolRegistry,
23    ): ...
24    def run(
25        self,
26        context: SkillContext,
27        system_prompt: str,
28        user_message: str,
29        fallback_tools: list[str] | None = None,
30    ) -> tuple[dict[str, Any], list[dict[str, Any]], dict[str,
31        Any]]: ...

```

Listing 17: 技能系统接口

10.8 辅助工具模块

```

1 # utils/prompt_loader.py
2 class PromptLoader:
3     """提示词加载器"""
4     def __init__(self, prompts_dir: Path): ...
5     def load(self, relative_path: str) -> str: ...
6
7 # utils/config_utils.py
8 def load_yaml_mapping(path: Path) -> dict[str, Any]: ...
9 def resolve_submit_config_presets(
10     config: dict[str, Any], presets: dict[str, Any]
11 ) -> dict[str, Any]: ...

```

```

12 def build_prompt_ensemble_yaml(
13     config: dict[str, Any], presets: dict[str, Any]
14 ) -> str: ...
15 def build_prompt_submit_config_yaml(
16     config: dict[str, Any], presets: dict[str, Any]
17 ) -> str: ...
18 def build_runtime_launch(runtime_cfg: dict[str, Any]) -> str: ...
19
20 # utils/io_utils.py
21 def read_text(path: Path) -> str: ...
22 def write_text(path: Path, content: str) -> None: ...
23 def write_json(path: Path, data: Any) -> None: ...
24 def extract_json_from_text(text: str) -> dict[str, Any]: ...
25
26 # utils/static_check.py
27 def analyze_bundle_static(
28     *,
29     main_program: str,
30     test_submit_script: str,
31     full_submit_script: str,
32 ) -> dict[str, Any]: ...
33 # 对生成的代码和脚本进行静态分析
34 # 返回: {main_errors, test_sh_errors, full_sh_errors}

```

Listing 18: 辅助工具函数

10.9 Wick 收缩生成器

```

1 # utils/generate_einsum/
2 from utils.generate_einsum import hadron_operator,
   _wick_translate
3
4 # hadron_operator.py — 强子插值算符定义
5 class HadronOperator: ...
6
7 # _wick_translate.py — Wick定理展开与einsum代码生成
8 def generate_contraction_einsum(
9     operator: HadronOperator, ...
10 ) -> str: ...

```

Listing 19: Wick 收缩与 einsum 生成

11 依赖关系

LQCD Master 依赖于以下组件：

表 5: 依赖关系

依赖	版本要求	说明
Python	≥ 3.10	核心语言
openai	≥ 1.0	OpenAI 兼容 API 客户端
prompt_toolkit	≥ 3.0	交互式命令行提示
PyYAML	≥ 6.0	YAML 配置解析
numpy	≥ 1.24	数值计算
opt_einsum	≥ 3.0	einsum 优化
cupy	≥ 12.0	GPU 加速 (CUDA/ROCm)
PyQUDA	最新	Python-QUDA 接口
QUDA	develop 分支	GPU 格点 QCD 库
Slurm	—	集群作业调度

12 开发与贡献

12.1 自定义集群配置

1. 编辑 `configs/config.yaml`: 设置 Slurm 分区、模块加载和 MPI 启动参数
2. 编辑 `configs/ensemble_presets.yaml`: 添加你的系综信息 (规范组态路径、夸克质量等)

12.2 添加新技能

1. 在 `skills/` 下创建新目录
2. 编写 `SKILL.md` 文件 (领域知识和 LLM 指令)
3. 在 `configs/skills.yaml` 中注册技能并配置路由规则

12.3 扩展基准测试

1. 在 `benchmark/` 中添加新任务定义
2. 提供手写的标准方法 `PyQUDA` 参考实现及数值输出
3. 运行交叉验证以评估新 LLM 骨干模型